

KB003 - Database Performance Incident Troubleshooting Runbook

Document Metadata

Field	Value
Document ID	KB003
Document Type	Database Operations Runbook / Known Error Document
Service Area	Database Infrastructure
Application Owner	Database Reliability Engineering Team
Support Group	Database SRE L2
Severity Coverage	P1, P2
Environment	Production, Staging
Last Updated	2026-01-25
Review Cycle	Quarterly

1. Overview

Database systems provide persistent storage capabilities for business-critical applications including:

- Payment Gateway
- Authentication Service
- Order Management
- Customer Profile Service
- Reporting Systems

Database performance incidents occur when applications cannot efficiently read or write data because of database resource exhaustion, slow queries, locking issues, or infrastructure failures.

Database degradation frequently appears as application-level failures such as:

- API timeout errors
- Slow page loading
- Failed transactions
- Login failures
- Increased application error rates

2. Common Incident Symptoms

Application Level Symptoms

Users may experience:

- Slow application response
- Failed API requests
- Transaction failures
- Login delays
- Data loading failures

Example errors:

```
HTTP 504 Gateway Timeout
Database request exceeded timeout
Unable to process request
```

Application logs:

```
ERROR DatabaseClient
DatabaseTimeoutException
Query execution exceeded configured timeout limit
```

Another common error:

```
ConnectionPoolException:
Unable to acquire database connection
```

3. Monitoring Indicators

During database incidents, verify these metrics.

Database Metrics

Metric	Warning Threshold	Critical Threshold
CPU Usage	>75%	>90%
Connection Usage	>70%	>95%
Query Latency P95	>2 sec	>5 sec
Disk Usage	>80%	>90%
Lock Wait Time	>10 sec	>30 sec

Important dashboards:

- Database health dashboard
- Query performance dashboard
- Application dependency dashboard
- Infrastructure monitoring

Metrics:

```
database.cpu.usage
```

```
database.connection.active
```

```
database.query.duration
```

```
database.lock.wait.time
```

```
database.storage.used
```

4. Common Root Causes and Resolution Steps

RCA Pattern 1: Long Running Database Queries

Description

Poorly optimized queries can consume database resources for extended periods.

These queries block other application requests and increase latency.

Common causes:

- Missing indexes
 - Large table scans
 - Inefficient joins
 - Poor query design
-

Symptoms

Application:

```
Application response time increases
```

```
API timeout errors observed
```

Database:

```
Query execution time > threshold
```

```
High CPU utilization
```

```
Increasing query queue
```

Logs:

```
SQLTimeoutException:
```

```
Query execution exceeded timeout limit
```

Validation Steps

Identify slow queries:

```
SELECT
query,
execution_time,
created_time

FROM query_history

ORDER BY execution_time DESC;
```

Check execution plan:

```
EXPLAIN ANALYZE
SELECT *
FROM transactions;
```

Look for:

```
Full table scan
```

```
Missing index
```

```
High cost query
```

Immediate Resolution

Actions:

1. Kill problematic queries.
2. Restart blocked sessions.
3. Increase query timeout temporarily if required.
4. Redirect traffic to healthy replicas.

Example:

```
KILL QUERY query_id;
```

Permanent Fix

Implement:

- Query optimization
 - Proper indexing
 - Query performance testing
 - Database monitoring alerts
-

RCA Pattern 2: Database Connection Pool Exhaustion

Description

Applications maintain a limited pool of database connections.

When all connections are occupied, new requests cannot communicate with the database.

Common reasons:

- Traffic spike
 - Connection leaks
 - Slow database queries
 - Incorrect pool configuration
-

Symptoms

Application logs:

```
Connection is not available
```

```
Request timed out waiting for database connection
```

Connection metrics:

```
active_connections = max_connections
```

```
connection_wait_time increasing
```

Validation

Check active database sessions:

```
SELECT COUNT(*)  
FROM active_connections;
```

Find idle connections:

```
SELECT *  
FROM database_sessions  
WHERE state='IDLE';
```

Immediate Resolution

Steps:

1. Restart affected application instances.
2. Remove stale database sessions.
3. Temporarily increase connection pool size.

Configuration example:

```
database:  
connectionPool:  
maximumPoolSize: 200  
connectionTimeout: 30000
```

Permanent Fix

Implement:

- Connection leak detection
 - Proper connection closing
 - Pool usage monitoring
 - Application code fixes
-

RCA Pattern 3: Database Locking Issue

Description

Database locks occur when one transaction prevents another transaction from accessing required data.

Severe locking issues can create application outages.

Symptoms

Application:

Save operation stuck

Transaction processing delayed

Database timeout errors

Database:

High lock wait time

Increasing blocked transactions

Validation

Find blocking transactions:

```
SELECT *  
FROM blocking_sessions;
```

Check lock duration:

```
SELECT *  
FROM database_locks  
ORDER BY lock_duration DESC;
```

Resolution

Immediate:

- Terminate blocking session
- Restart stuck transactions

Permanent:

- Reduce transaction duration
 - Optimize transaction logic
 - Improve database design
-

RCA Pattern 4: Storage Capacity Issue

Description

Database performance may degrade when available storage becomes critically low.

Causes:

- Excessive logs
 - Rapid data growth
 - Failed cleanup jobs
 - Large temporary files
-

Symptoms

Errors:

```
Database storage limit exceeded
Cannot write data
Transaction failed
```

Metrics:

```
disk_usage_percentage > 90%
```

Validation

Check storage:

```
df -h
```

Identify large tables:

```
SELECT
table_name,
size
FROM database_tables
ORDER BY size DESC;
```

Resolution

Immediate:

1. Increase database storage.
2. Remove temporary files.
3. Archive old data.

Permanent:

- Implement data retention policy
 - Configure storage alerts
 - Optimize database growth
-

RCA Pattern 5: Database Replica Lag

Description

Applications may use read replicas to improve performance.

If replicas fall behind primary databases, users may receive outdated data or failures.

Symptoms

Examples:

Recently updated data missing
Reports showing old information
Read requests failing

Metrics:

replication_lag_seconds increasing

Resolution

Immediate:

- Route reads to primary database
- Restart replica synchronization

Permanent:

- Improve replication monitoring
 - Scale replica capacity
-

5. Database Incident Investigation Checklist

Step 1: Identify Impact

Questions:

- Which applications are affected?
 - Read failure or write failure?
 - Complete outage or degraded performance?
-

Step 2: Check Recent Changes

Verify:

- Database deployments
 - Schema migrations
 - Configuration changes
 - Application releases
-

Step 3: Review Database Health

Check:

- CPU
 - Memory
 - Connections
 - Locks
 - Storage
 - Query latency
-

Step 4: Apply Recovery Action

Possible mitigations:

- Restart database connections
 - Scale resources
 - Kill blocking queries
 - Increase capacity
-

6. Escalation Matrix

Issue Type	Escalation Team
Query Performance	Database Engineering
Storage Issue	Infrastructure Team
Connection Leak	Application Development Team
Database Outage	Database Operations
Schema Issue	Application Engineering

7. Related Historical Incidents

INC-30521

Issue:

Payment Gateway experienced timeout errors.

Root Cause:

Database connection pool exhaustion.

Resolution:

Increased pool configuration and fixed connection leak.

INC-30845

Issue:

Application APIs became slow during peak traffic.

Root Cause:

Missing database index caused expensive queries.

Resolution:

Added index and optimized query.

INC-31102

Issue:

Order processing stopped intermittently.

Root Cause:

Database transaction locking.

Resolution:

Killed blocking sessions and optimized transaction handling.

8. RCA Generation Template

Database RCA should include:

Root Cause

Primary database failure mechanism.

Evidence

Include:

- Database metrics
- Application logs
- Related incidents
- Query analysis

Resolution Applied

Recovery steps performed.

Preventive Actions

Permanent improvements.

Example:

Root Cause:

Database connection pool exhaustion caused application timeout failures.

Evidence:

- Active connections reached maximum limit.
- Payment APIs returned timeout errors.
- Similar incident INC-30521 identified.

Resolution:

Restarted application workers and increased pool size.

Preventive Action:

Implemented connection monitoring and leak detection.

End of Document